

ServiceFlow — Documentação de Arquitetura

Projeto: atividade_flutter_n2
Aplicativo: ServiceFlow — Gestão de Ordens de Serviço
Versão do documento: 1.0
Data: maio/2026

1. O que é este aplicativo?

O **ServiceFlow** é um app mobile (Flutter) para técnicos e gestores registrarem e acompanharem **ordens de serviço (O.S.)**. Ele permite:

- Fazer login com conta segura (Supabase)
- Cadastrar clientes, técnicos, serviços e usuários
- Abrir, iniciar e finalizar ordens de serviço
- Tirar fotos e coletar assinatura digital como evidência
- Trabalhar **sem internet**: os dados ficam no celular e sincronizam quando a rede voltar

O foco da arquitetura é **reaproveitar código**: regras comuns ficam em classes base no núcleo (`core`), e cada funcionalidade (módulo) só implementa o que é específico.

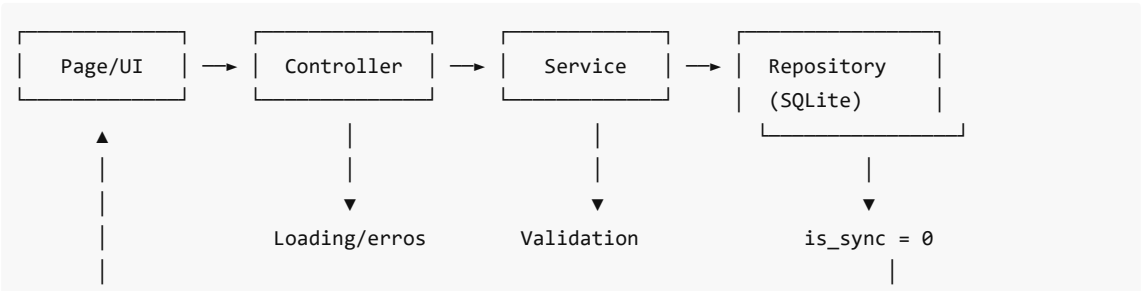
2. Visão geral da arquitetura

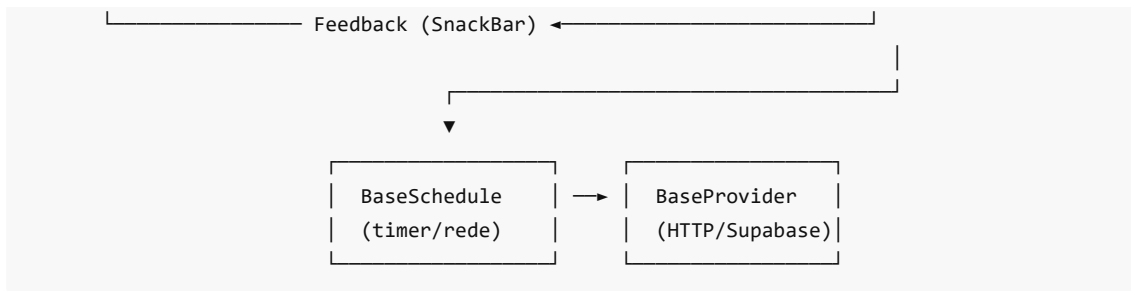
O projeto segue uma **arquitetura em camadas**, organizada por pastas:

Camada	Pasta	O que faz (em linguagem simples)
Entrada do app	<code>lib/main.dart</code>	Liga o Supabase, o banco local e a sincronização antes de abrir a tela
Interface global	<code>lib/app/shared/</code>	Rotas, tema e widgets reutilizáveis (botões, campos, cards)
Regras e dados	<code>lib/app/core/</code>	Modelos, validações, serviços, repositórios, HTTP, sync
Telas por função	<code>lib/app/modules/</code>	Login, dashboard, clientes, O.S., etc.

Fluxo de uma operação típica (ex.: salvar um cliente)

- Page (tela)** — o usuário preenche o formulário e toca em Salvar.
- Controller** — recebe a ação, mostra loading e trata erros para exibir mensagens.
- Service** — aplica validações de negócio e chama o repositório.
- Validation** — confere campos obrigatórios e regras (e-mail duplicado, etc.).
- Repository** — grava ou lê no **SQLite** (banco no aparelho).
- Schedule + Provider** — em segundo plano, quando há internet, envia para o **Supabase** na nuvem.





3. Estrutura de pastas

```
lib/
├─ main.dart                # Ponto de entrada
├─ app/
│   └─ core/                # Núcleo reutilizável
│       ├── base/           # Classes abstratas (Model, Repository, etc.)
│       ├── models/         # Entidades de negócio
│       ├── repositories/   # Acesso ao SQLite
│       ├── services/       # Regras de negócio
│       ├── validations/    # Validações por entidade
│       ├── controllers/    # Lógica das telas
│       ├── providers/      # Comunicação com API (nuvem)
│       ├── schedules/      # Sincronização automática
│       ├── http/           # Cliente Dio + interceptors
│       ├── helpers/        # Banco, sessão, config, sync
│       ├── logging/        # Registro de erros
│       ├── mixins/         # Loading e mensagens na UI
│       └─ theme/           # Cores e estilo visual
│   └─ shared/              # Rotas e widgets compartilhados
│   └─ modules/             # Telas por funcionalidade
│       ├── splash/
│       ├── auth/
│       ├── dashboard/
│       ├── clientes/
│       ├── tecnico/
│       ├── servico/
│       ├── usuario/
│       └─ ordem_servico/
```

4. Inicialização do aplicativo (`main.dart`)

Ao abrir o app, acontece nesta ordem:

1. **WidgetsFlutterBinding** — prepara o motor do Flutter.
2. **Supabase.initialize** — conecta ao backend de autenticação e API na nuvem.
3. **Somente em mobile** (não na web):
 - **DatabaseHelper** — abre ou cria o banco SQLite `serviceflow.db`.
 - **LogService** — prepara registro de erros.
 - **SyncSystemInitializer** — inicia os agendadores de sincronização.

4. `runApp(AppWidget)` — exibe o app com rotas e tema.

A tela inicial é a **Splash**, que redireciona para login ou dashboard conforme a sessão.

5. Classes base (o “esqueleto” do projeto)

Estas classes evitam repetir código em cada módulo.

5.1 BaseModel

Toda entidade de negócio herda dela.

Campo	Significado
<code>id</code>	Identificador do registro
<code>createdAt</code>	Data de criação
<code>isSync</code>	0 = ainda não foi para a nuvem; 1 = sincronizado
<code>ativo</code>	true = ativo; false = desativado (exclusão lógica)

Métodos: `toMap()` e construtor `fromMap()` para converter entre objeto Dart e linha do SQLite.

5.2 BaseRepository<E>

Centraliza operações no banco local: `insert`, `update`, `delete`, `findAll`, `findById`, `softDelete`, `findAllPendingSync`, `markAsSynced`.

Cada repositório concreto define `tableName` e `fromMap`.

5.3 BaseValidation<E, R>

Valida campos e regras antes de criar ou atualizar (implementado por entidade: `ClienteValidation`, etc.).

5.4 BaseService<E, R, V>

Orquestra o fluxo CRUD: chama validação → hooks `beforeCreate` / `afterCreate` → repositório.

5.5 BaseController<E, R, V, S>

Liga a tela ao serviço. Oferece:

- `isLoading` — indica carregamento na UI
- `executeOperation`, `executeListOperation`, `executeCrudOperation` — tratam erros de forma padronizada

5.6 BaseProvider<E>

Fala com a API externa (Supabase via REST): `syncToCloud`, `fetchFromCloud`, `deleteFromCloud`. Converte formato local ↔ formato da nuvem.

5.7 BaseSchedule<E, R, P>

Motor **offline-first**:

- Timer a cada 5 minutos (padrão)
- Detecta quando a internet volta (`connectivity_plus`)
- Envia registros com `is_sync = 0` para a nuvem
- Baixa atualizações remotas

5.8 Outras classes importantes no core

Classe	Função
AppWidget	MaterialApp com tema e rotas
AppRoutes	Mapa de rotas nomeadas
AppTheme	Tema visual claro do app
DatabaseHelper	Singleton do SQLite; executa <code>create_tables.sql</code>
AppClient	Cliente HTTP (Dio) para Supabase
AuthInterceptor	Coloca token JWT nas requisições
ErrorInterceptor	Padroniza erros de rede
AuthService	Login/logout com Supabase
SessionManager	Guarda token com <code>flutter_secure_storage</code>
ScheduleManager	Registra e inicia todos os schedules
SyncSystemInitializer	Ponto único para ligar/desligar sync
LogService	Log estruturado de falhas

6. Modelos de dados (entidades)

Modelo	Descrição principal
UsuarioModel	Usuário do sistema (nome, e-mail); id pode ser texto (Supabase)
ClienteModel	Cliente atendido (nome, documento, telefone, e-mail)
TecnicoModel	Técnico que executa o serviço (nome, especialidade)
ServicoModel	Tipo de serviço no catálogo (descrição, preço, tempo estimado)
OrdemServicoModel	Ordem de serviço: vínculos com cliente/técnico/serviço, datas, fotos, assinatura, status

Status da O.S. (`StatusOS`): Em Andamento, Finalizado, Cancelado.

Tabelas correspondentes no SQLite: `usuarios` , `clientes` , `tecnicos` , `servicos` , `ordens_servico` (definidas em `assets/sql/create_tables.sql`).

7. Módulos (telas e funcionalidades)

Módulo	Responsabilidade
--------	------------------

splash	Tela de abertura; verifica sessão
auth	LoginPage, RegisterPage — autenticação Supabase
dashboard	Painel do técnico: resumo, atalhos, lista de O.S., sync manual
clientes	Listagem, formulário e detalhes de clientes
tecnico	CRUD de técnicos
servico	CRUD do catálogo de serviços
usuario	Gestão de usuários
ordem_servico	Listar O.S., iniciar (foto antes), detalhes, finalizar (foto depois + assinatura)

Cada módulo segue o padrão: **Page** + **Controller** + dependências injetadas no construtor (service, validation, repository).

8. Sincronização offline-first

1. **Salvar sempre local primeiro** — o repositório grava no SQLite com `is_sync = 0`.
2. **ScheduleManager** registra um schedule por entidade: usuários, clientes, técnicos, serviços, ordens de serviço.
3. Quando há rede, o schedule:
 - **Upload:** envia pendentes via `BaseProvider.syncToCloud`
 - **Download:** busca novidades com `fetchFromCloud`
 - Marca `is_sync = 1` após sucesso
4. **Dashboard** pode forçar sync com `SyncSystemInitializer.forceSyncAll()`.

Na **web**, o banco local e o sync não são inicializados (`kIsWeb` em `main.dart`).

9. Rede e segurança

- **Supabase** — autenticação e API REST (`AppConfig` com URL e chave anônima).
- **Dio (`ApiClient`)** — todas as chamadas HTTP passam por interceptors.
- **Token** — armazenado de forma segura; o `AuthInterceptor` adiciona o header `Authorization`.
- Pacotes auxiliares: `image_picker`, `signature`, `mask_text_input_formatter`, `intl`.

10. Widgets compartilhados (`shared/widgets`)

Componentes visuais padronizados para manter a interface consistente:

- `CustomTextField`, `CustomButton`, `CustomElevatedButton`
- `CustomDropdown`, `CustomDatePicker`, `CustomCard`
- `CustomImagePicker` — seleção de fotos
- `AppLogo`

11. Dependências principais (`pubspec.yaml`)

Pacote	Uso
sqflite	Banco SQLite local
supabase_flutter	Auth e backend
dio	Cliente HTTP
provider	Estado (onde aplicável)
connectivity_plus	Detectar internet
flutter_secure_storage	Token seguro
image_picker / signature	Evidências na O.S.

12. Resumo para estudo ou apresentação

- O app é **modular**: cada pasta em `modules/` é uma funcionalidade.
- O **core** concentra a inteligência reutilizável (base classes + sync + HTTP).
- O padrão **Page** → **Controller** → **Service** → **Validation** → **Repository** se repete em todas as entidades.
- A nuvem entra pelo **Provider** + **Schedule**, sem a tela precisar saber detalhes de rede.
- **Offline-first**: o técnico trabalha no campo; a sincronização acontece depois, automaticamente ou manualmente.

Documento gerado com base no código-fonte do repositório `atividade_flutter_n2`.